

What is a Widget in Flutter?

In Flutter, **everything is a widget**.

A **widget** is a **basic building block of a Flutter app's user interface (UI)**. Buttons, text, images, layouts, and even the whole screen are widgets.

Simple definition:

A widget describes **how the UI should look and behave**.

Examples of widgets:

- Text
- Button
- Image
- Container
- Center
- Column
- Row

Types of Widgets in Flutter

There are mainly **two types of widgets**:

Stateless Widget

- Does **not change** during runtime
- Example: Text, Icon

Stateful Widget

- Can **change** when data changes
- Example: Checkbox, TextField

What is a Container Widget?

The **Container widget** is used to **store, style, and position other widgets**.

It is one of the **most commonly used widgets** in Flutter.

Think of Container as a **box** that can have:

- Height & width
- Color
- Border
- Padding & margin
- Alignment

Uses of Container Widget

- To give **background color**
- To set **size**
- To add **border or shadow**
- To add **spacing**

Example of Container Widget

```
Container(
  height: 100,
  width: 200,
  color: Colors.blue,
  child: Text("Hello Flutter"),
)
```

Explanation:

- height → height of container
- width → width of container
- color → background color
- child → widget inside container

What is Center Widget?

The **Center widget** places its child widget **exactly in the center of the screen or parent widget**.

It is mainly used for **alignment**.

Example of Center Widget

```
Center(
  child: Text("Welcome"),
)
```

Explanation:

- child → widget that needs to be centered

Container + Center Example

```
Center(
  child: Container(
    height: 100,
    width: 150,
    color: Colors.green,
    child: Center(
      child: Text("Center Text"),
    ),
  ),
)
```

What is Text Widget?

The **Text widget** is used to **display text on the screen**.

It is one of the **most basic and important widgets** in Flutter.

Example of Text Widget

```
Text("Hello Flutter")
```

Text Styling Example

```
Text(  
  "Flutter Learning",  
  style: TextStyle(  
    fontSize: 20,  
    color: Colors.red,  
    fontWeight: FontWeight.bold,  
  ),  
)
```

Explanation:

- `fontSize` → size of text
- `color` → text color
- `fontWeight` → bold or normal

Summary (For Students)

Widget	Purpose
Widget	Building block of Flutter UI
Container	Box for styling and layout
Center	Aligns widget to center
Text	Displays text

Key Points to Remember

- Flutter UI is made of **widgets**
- Widgets can be **nested inside each other**
- Container is used for **design and layout**
- Center is used for **alignment**
- Text is used to **show text**

What is a Button in Flutter?

A **button** is a widget used to **perform an action when the user clicks or taps on it**.

Example actions:

- Submit a form
- Navigate to next screen
- Show a message
- Save data

Types of Buttons in Flutter

ElevatedButton

- Most commonly used button
- Looks like a **raised button**
- Used for **important actions**

Example:

```
ElevatedButton(
  onPressed: () {
    print("Button Clicked");
  },
  child: Text("Click Me"),
)
```

Key Points:

- `onPressed` → action when button is clicked
- `child` → text or widget inside button

TextButton

- Looks like **plain text**
- No background color by default
- Used for **less important actions**

Example:

```
TextButton(
  onPressed: () {
    print("Text Button Clicked");
  },
  child: Text("Text Button"),
)
```

OutlinedButton

- Button with **border only**
- No background color

Example:

```
OutlinedButton(
  onPressed: () {
    print("Outlined Button Clicked");
  },
)
```

```
    },  
    child: Text("Outlined"),  
  )  
)
```

IconButton

- Button with an **icon**
- Used mostly in **AppBar**

Example:

```
IconButton(  
  icon: Icon(Icons.add),  
  onPressed: () {  
    print("Icon Button Clicked");  
  },  
)
```

Disabled Button

If `onPressed` is **null**, the button becomes disabled.

```
ElevatedButton(  
  onPressed: null,  
  child: Text("Disabled"),  
)
```

What is Image Widget?

The **Image widget** is used to **display images** in a Flutter app.

Flutter supports images from:

1. Assets (project images)
2. Network (internet images)

Types of Images in Flutter

1. Image from Assets (Local Image)

Step 1: Create an Images Folder

Inside your Flutter project:

```
assets/images/
```

Put image inside this folder:

```
assets/images/logo.png
```

Step 2: Add Image Path in pubspec.yaml

```
flutter:
  assets:
    - assets/images/logo.png
```

Important:

- Proper indentation is very important
- Run `flutter pub get` after saving

Step 3: Use Image in Code

```
Image.asset(
  'assets/images/logo.png',
  width: 150,
  height: 150,
)
```

2. Image from Network (Internet Image)

Example:

```
Image.network(
  'https://flutter.dev/images/flutter-logo-sharing.png',
  width: 150,
)
```

Using Image inside Container

```
Container(
  height: 200,
  width: 200,
  child: Image.asset(
    'assets/images/logo.png',
    fit: BoxFit.cover,
  ),
)
```

BoxFit types:

- `cover` → fills container
- `contain` → keeps full image visible
- `fill` → stretches image

Button + Image Example

```
Column(
  mainAxisAlignment: MainAxisAlignment.center,
  children: [
    Image.asset('assets/images/logo.png', width: 100),
    SizedBox(height: 20),
    ElevatedButton(
      onPressed: () {
        print("Image Button Clicked");
      },
      child: Text("Press Me"),
    ),
  ],
)
```

```
],  
)
```

Summary Table

Widget	Purpose
ElevatedButton	Main action button
TextButton	Simple text button
OutlinedButton	Border button
IconButton	Icon-based button
Image.asset	Show local image
Image.network	Show online image

Important Notes for Students

- Buttons need `onPressed` to work
- Images must be declared in `pubspec.yaml`
- Asset images work **offline**
- Network images need **internet**

What are Row and Column Widgets?

Row and **Column** are **layout widgets** in Flutter.

They are used to **arrange widgets in a straight line**.

Main difference:

- **Row** → arranges widgets **horizontally** (left to right)
- **Column** → arranges widgets **vertically** (top to bottom)

What is a Row Widget?

A **Row widget** displays its child widgets **in a horizontal direction**.

Note: mainAxisAlignment: Controls **horizontal alignment** in Row.

```
Row(  
  mainAxisAlignment: MainAxisAlignment.spaceAround,  
  children: [  
    Icon(Icons.star),  
    Icon(Icons.star),  
    Icon(Icons.star),  
  ],  
)
```

Common values:

- `start`

- center
- end
- spaceBetween
- spaceAround
- spaceEvenly

Note: crossAxisAlignment: Controls **vertical alignment** in Row.

```
Row(
  crossAxisAlignment: CrossAxisAlignment.center,
  children: [
    Text("A"),
    Text("B"),
  ],
)
```

What is a Column Widget?

A **Column widget** displays its child widgets **vertically**.

Note: mainAxisAlignment: Controls **vertical alignment** in Column.

```
Column(
  mainAxisAlignment: MainAxisAlignment.center,
  children: [
    Text("Top"),
    Text("Middle"),
    Text("Bottom"),
  ],
)
```

Note: crossAxisAlignment: Controls **horizontal alignment** in Column.

```
Column(
  crossAxisAlignment: CrossAxisAlignment.start,
  children: [
    Text("Left"),
    Text("Aligned"),
  ],
)
```

MainAxis vs CrossAxis (Very Important)

Widget	Main Axis	Cross Axis
Row	Horizontal	Vertical
Column	Vertical	Horizontal

Row & Column with Container Example

```
Row(
  mainAxisAlignment: MainAxisAlignment.spaceEvenly,
  children: [
    Container(height: 50, width: 50, color: Colors.red),
```



```

        Container(height: 50, width: 50, color: Colors.green),
        Container(height: 50, width: 50, color: Colors.blue),
      ],
    ),
  ),
)

```

Nested Row and Column

You can put a **Row** inside a **Column** and vice-versa.

```

Column(
  children: [
    Text("Header"),
    Row(
      mainAxisAlignment: MainAxisAlignment.center,
      children: [
        Icon(Icons.home),
        SizedBox(width: 10),
        Text("Home"),
      ],
    ),
  ],
)

```

SizedBox for Spacing

Used to give **space between widgets**.

```

SizedBox(width: 20) // for Row
SizedBox(height: 20) // for Column

```

Common Errors

- **Overflow error** when Row has too many widgets
Solution: Use `Expanded` or `Wrap`
- Forgetting `children` list
- Confusing `mainAxis` and `crossAxis`

Summary Table

Widget	Direction	Use
Row	Horizontal	Place widgets side-by-side
Column	Vertical	Place widgets one below another

Key Points to Remember

- Row = horizontal layout
- Column = vertical layout
- `mainAxisAlignment` controls **direction of layout**
- `crossAxisAlignment` controls **opposite direction**
- Widgets can be nested

InkWell Widget & ScrollView Widgets

What is InkWell?

InkWell is a widget used to **detect touch (tap) events** and show a **ripple (water) effect** when the user taps on a widget.

It is mostly used to make **any widget clickable** (Container, Text, Image).

Why use InkWell?

- To add **tap functionality**
- To show **material ripple effect**
- To make UI look **interactive**

Basic InkWell Example

```
InkWell(  
  onTap: () {  
    print("Tapped");  
  },  
  child: Text("Tap Me"),  
)
```

InkWell Important Properties

Property	Use
onTap	Action on tap
onDoubleTap	Action on double tap
onLongPress	Action on long press
splashColor	Ripple color
borderRadius	Rounded ripple

InkWell with Border Radius

```
InkWell(  
  borderRadius: BorderRadius.circular(10),  
  onTap: () {},  
  child: Container(  
    padding: EdgeInsets.all(15),  
    decoration: BoxDecoration(  
      color: Colors.green,  
      borderRadius: BorderRadius.circular(10),  
    ),  
    child: Text("Tap"),  
  ),  
)
```

Important Note

InkWell works **best inside Material widget**.

If ripple is not visible, wrap with `Material`.

```
Material(  
  child: InkWell(  
    onTap: () {},  
    child: Container(...),  
  ),  
)
```

ScrollView Widgets in Flutter

A **ScrollView** allows content to **scroll when it does not fit on the screen**.

Used when UI content is **bigger than screen size**.

SingleChildScrollView

When to use?

- When content is **small to medium**
- Forms, profile pages, settings pages

Vertical Scroll (Default)

```
SingleChildScrollView(  
  child: Column(  
    children: [  
      Text("Item 1"),  
      Text("Item 2"),  
      Text("Item 3"),  
    ],  
  ),  
)
```

Horizontal Scroll Example

```
SingleChildScrollView(  
  scrollDirection: Axis.horizontal,  
  child: Row(  
    children: [  
      Container(width: 100, height: 100, color: Colors.red),  
      Container(width: 100, height: 100, color: Colors.green),  
      Container(width: 100, height: 100, color: Colors.blue),  
    ],  
  ),  
)
```

ListView (Most Used Scroll Widget)

When to use?

- Large lists

- Dynamic data
- Efficient scrolling

Vertical ListView

```
ListView(
  children: [
    ListTile(title: Text("Item 1")),
    ListTile(title: Text("Item 2")),
    ListTile(title: Text("Item 3")),
  ],
)
```

Horizontal ListView

```
ListView(
  scrollDirection: Axis.horizontal,
  children: [
    Container(width: 120, color: Colors.orange),
    Container(width: 120, color: Colors.purple),
    Container(width: 120, color: Colors.blue),
  ],
)
```

GridView

When to use?

- Images
- Products
- Cards layout

GridView Example

```
GridView.count(
  crossAxisCount: 2,
  children: [
    Container(color: Colors.red),
    Container(color: Colors.green),
    Container(color: Colors.blue),
    Container(color: Colors.orange),
  ],
)
```

Scroll Direction Summary

Widget	Vertical	Horizontal
SingleChildScrollView	Default	Use <code>Axis.horizontal</code>
ListView	Default	Use <code>Axis.horizontal</code>
GridView	Yes	Yes

Real-Life Example

Instagram App

- **Vertical Scroll** → Posts feed
- **Horizontal Scroll** → Stories

Common Beginner Mistakes

- Using `Column` instead of `ListView` for long lists
- Using `SingleChildScrollView` for large data
- Forgetting `scrollDirection`

Key Points to Remember

- `InkWell` makes widgets **clickable**
- `ScrollView` is needed when content overflows
- Use **`ListView`** for large lists
- Use **`SingleChildScrollView`** for forms
- Horizontal scroll needs `Axis.horizontal`

ListView Widgets in Flutter

`ListView` is a scrollable widget used to **display items in a vertical or horizontal list**.

Use `ListView` when you have **more content than screen height**.

ListView (Static List)

When to use?

- Small number of items
- Fixed (static) data
- Simple UI

Example: Simple ListView

```
ListView(  
  children: [  
    ListTile(title: Text("Neeraj")),  
    ListTile(title: Text("Aman")),  
    ListTile(title: Text("Rohan")),  
  ],  
)
```

Pros

- Easy to use
- Good for small lists

Cons

- Not memory efficient for large lists

ListView.builder (Dynamic List)

ListView.builder creates list items **only when they are visible on the screen**.

This makes it **fast and memory-efficient**.

When to use?

- Large lists
- Data from API / database
- Dynamic content

Example: List of 5 Students (Requested Example)

Step 1: Create Student List

```
List<String> students = [  
    "Neeraj",  
    "Aman",  
    "Rohan",  
    "Shyam",  
    "Mohan",  
];
```

Step 2: ListView.builder Code

```
ListView.builder(  
    itemCount: students.length,  
    itemBuilder: (context, index) {  
        return ListTile(  
            leading: CircleAvatar(  
                child: Text(students[index][0]),  
            ),  
            title: Text(students[index]),  
            subtitle: Text("Student"),  
        );  
    },  
)
```

Explanation:

- `itemCount` → total number of items
- `itemBuilder` → builds each item dynamically
- `index` → position of item (0,1,2...)

ListView.separated

This ListView allows you to **add a separator** between list items.

Mostly used to add **Divider or spacing**.

When to use?

- When you need a line or space between items
- Settings pages
- Contact lists

Example: `ListView.separated`

```
ListView.separated(  
  itemCount: students.length,  
  itemBuilder: (context, index) {  
    return ListTile(  
      title: Text(students[index]),  
    );  
  },  
  separatorBuilder: (context, index) {  
    return Divider(  
      thickness: 1,  
      color: Colors.grey,  
    );  
  },  
)
```

Horizontal `ListView.builder` Example

```
ListView.builder(  
  scrollDirection: Axis.horizontal,  
  itemCount: students.length,  
  itemBuilder: (context, index) {  
    return Container(  
      width: 120,  
      margin: EdgeInsets.all(8),  
      color: Colors.blue,  
      child: Center(  
        child: Text(  
          students[index],  
          style: TextStyle(color: Colors.white),  
        ),  
      ),  
    );  
  },  
)
```

Real-Life Example

WhatsApp:

- Chat list → **`ListView.builder`**
- Contact list → **`ListView.separated`**

Common Beginner Mistakes

- Using `Column` instead of `ListView`
- Forgetting `itemCount`
- Putting `ListView` inside `Column` without `Expanded`

Summary Table

Widget	Use
ListView	Small static list
ListView.builder	Large dynamic list
ListView.separated	List with dividers

Key Points to Remember

- Use **ListView.builder** for performance
- Use **ListView.separated** for dividers
- ListView is scrollable by default
- Always give `itemCount`